

M09 Teaching Book

Beginner-first, production-minded transformation engineering. Use the assigned visual to see the pattern, then close it and prove the course-specific behavior yourself.

The local executable lab is the authority for deterministic counts/state behavior. Official docs are the authority for current dbt/Confluent semantics. Videos are visual teachers, not answer keys.

DE09-01 - ETL vs ELT and transformation boundaries

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - Introduction to Analytics Engineering](#)

Whole introduction; use it for ETL/ELT and analytics-engineering context.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Choose ETL vs ELT boundaries from trust, compute and ownership rather than slogans.

Teach it from zero

ETL transforms before final load; ELT lands source-shaped data first and transforms inside the destination. Both need contracts, lineage and reconciliation.

Teacher demonstration

Draw source -> landing/raw -> validated staging -> intermediate -> serving. Mark where each rule is enforced.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Given a CSV plus a warehouse, justify which cleanup must happen before load and which business modeling can happen after load.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If you argue only from tool preference, restate the data-risk, recoverability and ownership requirements.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[DataTalksClub 2026 course map](#)

MASTER only after a changed task works without the lesson/review open.

DE09-02 - Raw, staging, clean/intermediate and serving contracts

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Project Structure](#)

Project layers and dependency structure; course lab owns the exact layer contracts.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Define explicit contracts for raw, staging, intermediate and serving layers.

Teach it from zero

Raw preserves received evidence. Staging standardizes types/keys while surfacing bad rows. Intermediate expresses reusable business logic. Serving exposes stable consumer grain.

Teacher demonstration

Inspect the RetailPulse SQLite tables and lineage.json; state grain and allowed mutations at each layer.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Write the four layer contracts before writing transformations.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If layers blur together, ask: can I reconstruct source evidence; where is invalid data visible; what table is safe for consumers?

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt projects](#)

[dbt sources](#)

MASTER only after a changed task works without the lesson/review open.

DE09-03 - Deterministic transformations and production SQL style

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Models](#)

Deterministic model/dependency mental model.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Write deterministic transformations whose output is a function of declared inputs and rules.

Teach it from zero

Deterministic work avoids hidden manual edits, unstable ordering and ambient state. Production SQL should make grain, filters, joins and calculations readable.

Teacher demonstration

Run the baseline twice from a fresh database and compare serving controls.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Rewrite one ambiguous transformation as explicit typed/filter logic and add an independent control query.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If reruns differ, isolate nondeterminism: time-dependent defaults, unordered tie selection, mutable inputs or hidden state.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt models](#)

MASTER only after a changed task works without the lesson/review open.

DE09-04 - Deduplication and key integrity

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Tests](#)

Support for uniqueness/not-null/relationship thinking; dedup implementation is course-owned.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Separate exact duplicate removal from business-key conflict resolution.

Teach it from zero

An exact duplicate repeats the same received row. A repeated business key with changed attributes is not automatically a duplicate; choose a version/merge rule and prove it.

Teacher demonstration

RetailPulse contains one exact R10 duplicate. The pipeline quarantines it while preserving source evidence.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Prove raw received rows, exact duplicate count and one-current-row order_id in staging/serving.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

Never add DISTINCT to silence unexpected fan-out. Identify the key/grain first.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt data tests](#)

MASTER only after a changed task works without the lesson/review open.

DE09-05 - Schema checks and data contracts

VIDEO FIRST route

[Confluent - Data Contracts for Long-Term Schema Management](#)

Use contract/schema concepts; batch schema enforcement and quarantine are course-owned.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Enforce a schema/data contract and quarantine incompatible rows without silently coercing them.

Teach it from zero

A contract includes required fields, types, domain rules and evolution expectations. Compatibility decisions should be explicit.

Teacher demonstration

R11 has bad date and R12 misses customer_id; both remain visible in quarantine.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Add one safe optional-field evolution proposal and one breaking-change example.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If a cast fails, preserve the raw value and reason; do not convert every error to zero/null.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[Confluent schemas and data contracts](#)

MASTER only after a changed task works without the lesson/review open.

DE09-06 - Source-to-target reconciliation

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Sources](#)

Sources/freshness context; reconciliation is proven in the local lab.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Reconcile source, staging and serving with row and value controls.

Teach it from zero

Counts alone can pass while money is wrong; sums alone can pass while rows are duplicated. Reconciliation needs both plus exclusion rules.

Teacher demonstration

Baseline: 14 received, 11 accepted unique, 3 quarantined, 10 completed serving rows, revenue 2990.00.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Write controls for source rows, accepted rows, quarantined rows, serving rows, serving revenue and Cancelled exclusion.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If totals disagree, compare layer by layer until the first divergence; repair there.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt sources](#)

MASTER only after a changed task works without the lesson/review open.

DE09-07 - Incremental processing and safe reruns

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

Incremental-loading visual; course lab proves safe rerun/state behavior.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Process new/changed records incrementally and make reruns safe.

Teach it from zero

Incremental models need a trustworthy cursor and unique key. Reprocessing the same batch must not create duplicates.

Teacher demonstration

Apply R6 update + R14 new record; serving becomes 11 rows / 3490.00. Rerun selects 0.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Explain why updated_at is the cursor and order_id is the merge key; show rerun evidence.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If a changed key duplicates, check unique-key logic and >=</> cursor boundary before changing data.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt incremental models](#)

[dlt incremental loading](#)

MASTER only after a changed task works without the lesson/review open.

DE09-08 - Failure behavior, retries, quarantine and atomic state

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Commands](#)

Build/command support only; failure atomicity is proven by the course failure injection.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Keep destination and checkpoint/state atomic when a batch fails validation.

Teach it from zero

State must advance only after the destination commit is durable. Retry transient work; quarantine record-level defects; fail the run for contract-breaking conditions according to policy.

Teacher demonstration

Failure batch contains a numeric defect. `fail_on_quarantine=True` returns `FAILED_VALIDATION` and leaves watermark unchanged.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Inject one failure and prove destination row count and state are unchanged.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If state moved on failure, treat it as a correctness bug, not a logging issue.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt commands](#)

MASTER only after a changed task works without the lesson/review open.

DE09-09 - Lineage, run metadata and auditability

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Documentation](#)

Documentation/lineage context; run metadata is generated locally.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Produce lineage and run metadata sufficient to explain what ran and what changed.

Teach it from zero

Auditability links source file, run id, layer, counts, state before/after, status and output controls. Lineage records dependencies and grains.

Teacher demonstration

Inspect run_audit plus lineage.json produced by the pipeline.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Trace R6 from source update through staging to serving and identify its run metadata.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If you cannot answer which input/run produced a row, the handoff is incomplete.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt documentation](#)

MASTER only after a changed task works without the lesson/review open.

DE09-10 - Build and defend a reliable batch pipeline

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Project Structure](#)

Review structure -> sources -> models -> tests -> docs, then close the video and build.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Build and defend a reliable batch pipeline end to end.

Teach it from zero

A reliable batch path is not just transformation code: contracts -> validation -> quarantine -> deterministic transform -> reconciliation -> tests -> atomic state -> metadata -> rerun proof.

Teacher demonstration

Run m09_pipeline.py from a clean output folder and read M09_SMOKE_RESULT.json.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Rebuild the pipeline on a changed file and produce an evidence packet, not only a success message.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

When a result is plausible but controls fail, trust controls and investigate.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[DataTalksClub analytics engineering module](#)

MASTER only after a changed task works without the lesson/review open.

DE09-11 - dbt-style project anatomy, sources and dependency graph

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Project Structure](#)

Whole focused project-anatomy lesson.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Read a dbt-style project as a dependency graph instead of disconnected SQL files.

Teach it from zero

source() represents declared external/source relations; ref() creates model dependencies; project/YAML config adds metadata/tests/materialization behavior.

Teacher demonstration

Open dbt_style_project: source -> stg_orders -> int_completed_orders -> fct_sales.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Sketch the DAG, identify each model grain and explain why direct raw -> mart logic is harder to govern.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If the graph is unclear, follow source()/ref() edges before reading every SQL expression.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt projects](#)

[dbt sources](#)

MASTER only after a changed task works without the lesson/review open.

DE09-12 - Executable data tests, quality severity and CI boundaries

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Tests](#)

Whole focused tests lesson; course lab adds severity/CI boundaries.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Turn quality expectations into executable tests with severity and CI boundaries.

Teach it from zero

Blocking tests protect invariants such as required keys/uniqueness. Warning tests surface risk that may not justify stopping every run. CI should fail only on deliberately blocking conditions.

Teacher demonstration

The SQLite smoke runs unique key, required customer, positive revenue and reconciliation tests.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Classify five proposed checks as BLOCK or WARN and defend the business consequence.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

Do not make every anomaly fatal and do not make critical integrity checks warnings.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt data tests](#)

MASTER only after a changed task works without the lesson/review open.

DE09-13 - Incremental models, materialization, documentation and handoff

VIDEO FIRST route

[DataTalksClub DE Zoomcamp - dbt Models](#)

Models/materialization support; pair with current dbt incremental-model documentation.

Starting state / prerequisite

M08 ingestion mastery plus SQL/Python fundamentals. Keep the protected review closed until a genuine attempt is saved.

Outcome

Explain materialization/incremental behavior and hand off a reproducible project.

Teach it from zero

Views, tables and incremental models trade freshness, compute and persistence. Current dbt incremental models require valid SQL for both full/incremental paths; unique_key supports updates instead of blind append.

Teacher demonstration

Inspect fct_sales.sql with materialized=incremental, unique_key=order_id and on_schema_change=fail; compare it with the executed SQLite equivalent.

Guided reproduction

Use the supplied RetailPulse files and scripts. Reproduce the demonstrated control once, then close this page and repeat the same reasoning on a slightly changed slice.

Independent task

Write a handoff: inputs, grain, model DAG, tests, state/cursor, full-refresh/backfill decision, known limitations and rerun commands.

Validate

Save source/layer grain, row counts, key checks, at least one value reconciliation, run/state evidence where relevant, and one failure-mode note. A success message alone is not evidence.

Recover

If incremental logic changes historical meaning, plan a controlled rebuild/backfill rather than assuming future runs repair history.

Explain back

In 60-90 seconds explain the rule, one failure mode, one validation and why the chosen boundary protects correctness.

Current authoritative reference(s)

[dbt incremental models](#)

[dbt materializations](#)

[dbt documentation](#)

MASTER only after a changed task works without the lesson/review open.